

cs280_hw1

Ziyu Ge

February 2026

1 Poor Man's Augmented Reality

video result link:

<https://drive.google.com/file/d/1rPQdUPw6yWLRlUuWROqUlvTruQnDO-M/view?usp=sharing>

2 Affine Transforms & Homographies

2.1

2.1.1 (b) Solving for H in `affine_solve(u,v)`

Given $u, v \in \mathbb{R}^{2 \times N}$ representing N corresponding points, I restrict the affine transform to **rotation + translation only**. Thus, the transformation has the form:

$$\begin{aligned}x' &= cx - sy + t_x \\y' &= sx + cy + t_y\end{aligned}$$

where $c = \cos \theta$, $s = \sin \theta$, and t_x, t_y are translation parameters.

For each correspondence $(x_i, y_i) \rightarrow (x'_i, y'_i)$, I obtain two linear equations:

$$\begin{aligned}x'_i &= cx_i - sy_i + t_x \\y'_i &= sx_i + cy_i + t_y\end{aligned}$$

Stacking all N correspondences, I obtain a linear system:

$$A\mathbf{p} = \mathbf{b}$$

where

$$\mathbf{p} = \begin{bmatrix} c \\ s \\ t_x \\ t_y \end{bmatrix}$$

and $A \in \mathbb{R}^{2N \times 4}$, $\mathbf{b} \in \mathbb{R}^{2N}$.

Since the system may be overdetermined, solve using least squares:

$$\mathbf{p} = (A^\top A)^{-1} A^\top \mathbf{b}$$

The affine transformation matrix is then:

$$H = \begin{bmatrix} c & -s & t_x \\ s & c & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

This matrix is returned by `affine_solve(u,v)`.

2.1.2 (c) Applying the Transform in `do_transform(u,H)`

Given $u \in \mathbb{R}^{2 \times N}$ and $H \in \mathbb{R}^{3 \times 3}$, apply the transformation in homogeneous coordinates. augment each point with a 1:

$$\tilde{u} = \begin{bmatrix} u \\ \mathbf{1} \end{bmatrix} \in \mathbb{R}^{3 \times N}$$

$$\tilde{v} = H\tilde{u}$$

Since homogeneous coordinates are defined up to scale, divide by the third row:

$$v = \begin{bmatrix} \tilde{v}_x \\ \tilde{v}_y \\ \tilde{v}_z \end{bmatrix}$$

The resulting matrix $v \in \mathbb{R}^{2 \times N}$ is returned.

2.2 solve for H in a homography

now estimate a full planar homography $H \in \mathbb{R}^{3 \times 3}$ that maps points from the source image to a selected planar surface in the target image.

Using homogeneous coordinates, the mapping is defined as:

$$\tilde{v} = H\tilde{u}, \quad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}.$$

For a point (x, y) in the source image:

$$\begin{aligned} x' &= \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}, \\ y' &= \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}. \end{aligned}$$

The homography has 8 degrees of freedom (the scale is fixed by setting $h_{33} = 1$). Rearranging the equations gives two linear constraints per correspondence:

$$\begin{aligned} x'(h_{31}x + h_{32}y + 1) &= h_{11}x + h_{12}y + h_{13}, \\ y'(h_{31}x + h_{32}y + 1) &= h_{21}x + h_{22}y + h_{23}. \end{aligned}$$

These can be rewritten in linear form:

$$A\mathbf{h} = \mathbf{b},$$

where

$$\mathbf{h} = [h_{11} \ h_{12} \ h_{13} \ h_{21} \ h_{22} \ h_{23} \ h_{31} \ h_{32}]^\top.$$

Stacking all N correspondences yields a system $A \in \mathbb{R}^{2N \times 8}$ and $\mathbf{b} \in \mathbb{R}^{2N}$.

When $N \geq 4$, the system can be solved using least squares:

$$\mathbf{h} = (A^\top A)^{-1} A^\top \mathbf{b}.$$

The homography matrix is then reconstructed as:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}.$$

To transform all source image points, reuse the previously defined function `do_transform(u, H)`.

Each point is first converted to homogeneous coordinates:

$$\tilde{u} = \begin{bmatrix} u \\ \mathbf{1} \end{bmatrix},$$

then transformed:

$$\tilde{v} = H\tilde{u},$$

and finally converted back to Cartesian coordinates:

$$v = \begin{bmatrix} \frac{\tilde{v}_x}{\tilde{v}_z} \\ \frac{\tilde{v}_y}{\tilde{v}_z} \end{bmatrix}.$$

The transformed points are then used to construct the modified target image.

2.3

(c) Constraints on H for affine transform vs. homography For the affine transform (restricted to rotation and translation), the transformation matrix has the constrained form:

$$H_{\text{affine}} = \begin{bmatrix} c & -s & t_x \\ s & c & t_y \\ 0 & 0 & 1 \end{bmatrix},$$

where $c = \cos \theta$, $s = \sin \theta$, and t_x, t_y are translation parameters. Thus, the matrix has only 3 degrees of freedom: one rotation angle and two translations. The last row is fixed to $[0 \ 0 \ 1]$, and no scaling, shear, or perspective distortion is allowed.

In contrast, a homography has the more general form:

$$H_{\text{homography}} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}.$$

Here, the matrix has 8 degrees of freedom (since it is defined up to scale). The third row is not constrained to $[0 \ 0 \ 1]$, allowing projective effects. This enables perspective distortion and mapping between arbitrary planar surfaces.

(d) Can the affine transform exactly map the points? In general, the affine transform cannot exactly map the points from one image to another when the target surface involves perspective effects.

An affine transformation preserves:

- Parallelism of lines
- Ratios of distances along parallel directions

However, perspective projection does not preserve parallelism in the image plane. For example, parallel lines in 3D may converge toward a vanishing point.

Since the affine model does not include the projective terms (h_{31}, h_{32}), it cannot model this perspective distortion. Therefore, it cannot exactly align points when the surface is viewed under perspective projection.

A homography, on the other hand, includes the necessary projective terms and can exactly map points between two views of the same planar surface.